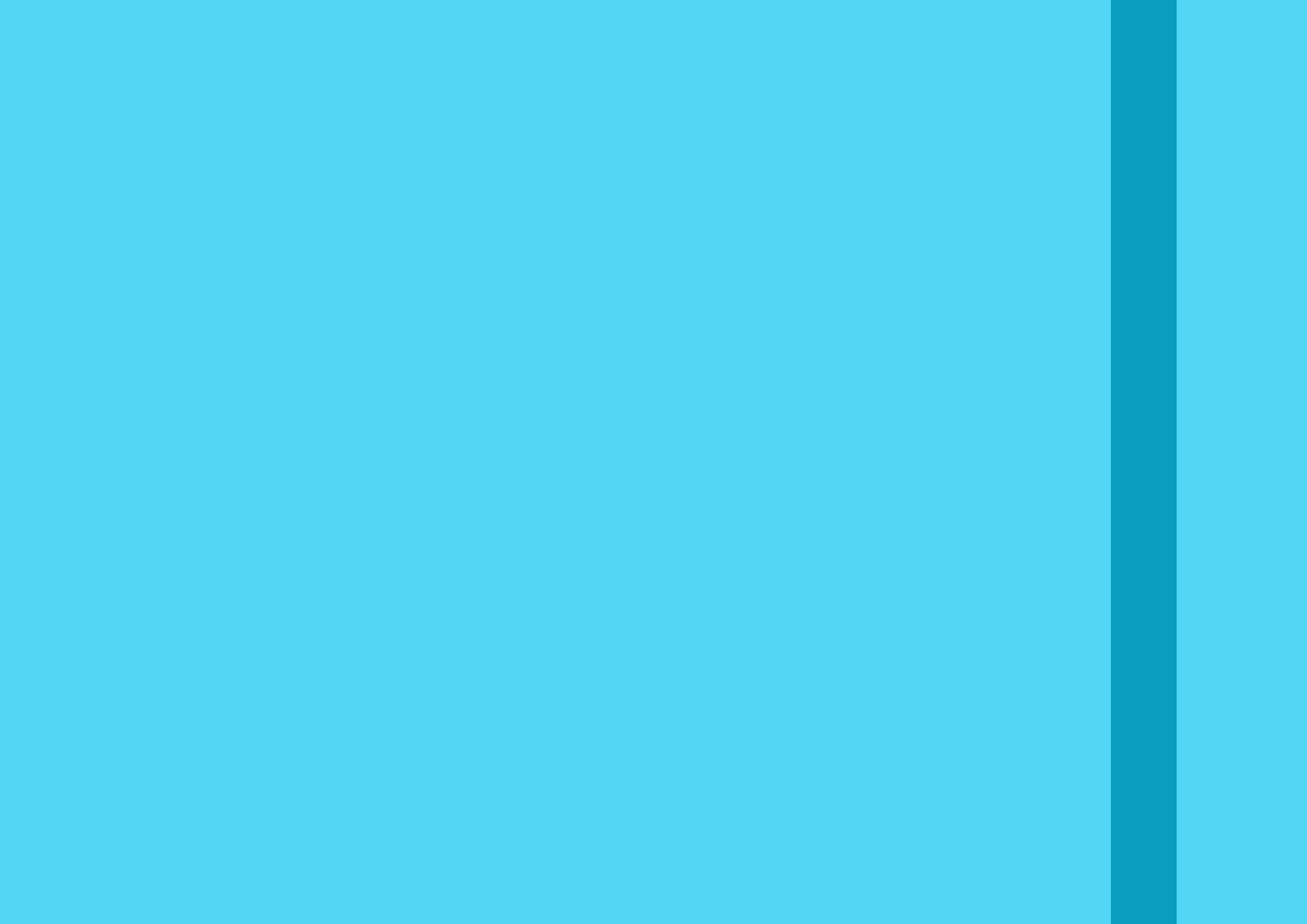




Java

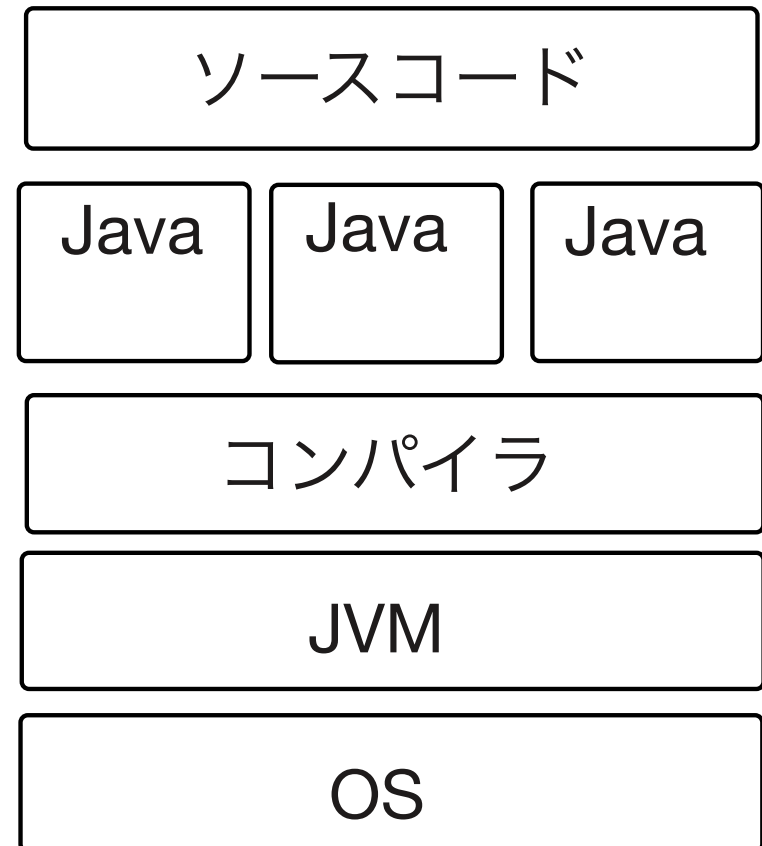
JRE: APIとJVMが内蔵

コンパイラがないため実行出来ない
(コンパイル出来ない)

JDK: APIとコンパイラ、JVMが内蔵

これだけで開発可能

Javaファイルをclassファイルへ
(ソースコードをバイトコードへ)



Spring Boot の特徴

- ・ ライブラリの一括取得 (Starter 依存関係の解決)
- ・ Auto Configuration

↳ Spring Boot が用意した Java Config を読む

① ライブラリが提供する 7つの Bean 定義

ex) Tomcat や Spring MVC, Thymeleaf などの Bean 生成

② Spring が提供する各機能の有効化

⇒ Spring Boot を使うと Java Config の Bean 定義と Annotation の 負担 を軽減することが出来る

• Auto Configuration

設定を反映させるのに必要なファイル

• application.properties or application.yml

↑ フォルダ直下に配置

・ Spring Boot の起動方法

① main メソッド内で SpringApplication.run を呼び出す
↳ application.properties を読み込む

② main メソッドで定義する @SpringBootApplication を追加する

@SpringBootApplication は以下 Annotation を含む

- ・ @Configuration
- ・ @EnableAutoConfiguration
- ・ @ComponentScan

• コンフィグレーション (@Configuration)

- DIのコンポーネント スキャンの範囲を定義 (@ComponentScan)

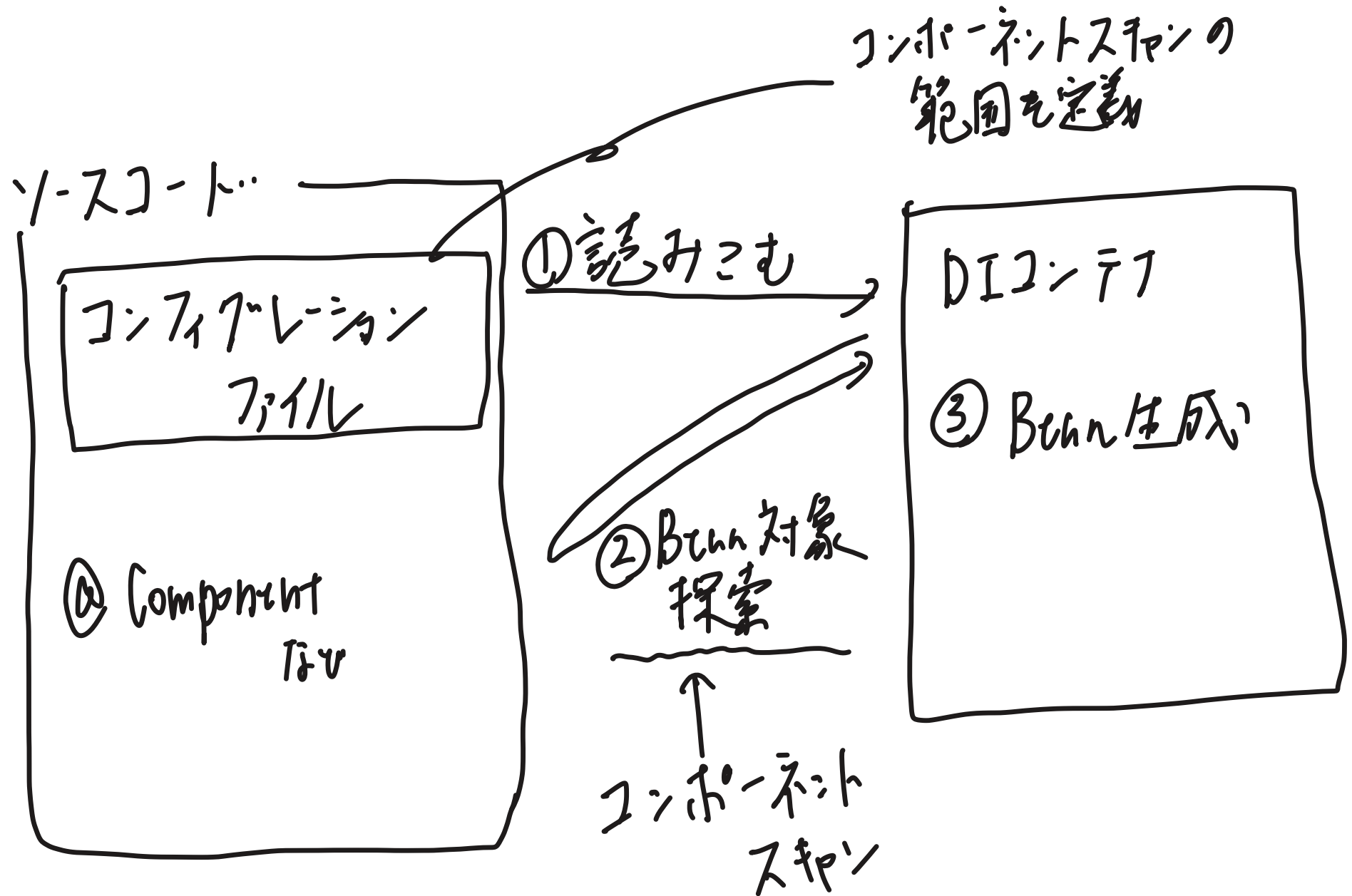
↑ コンフィグレーションファイルが属するパッケージ以下
or

特定のパッケージを指定する

- Bean定義 (Aト部ライブラリでBeanに登録が必要なものを)
(@Beanメソッドで登録(定義)する)

Beanに関する設定

■ DIコンテナー コンポーネントスキャン イメージ図



データベースアクセス方法

- ・ Spring Data JPA: O/Rマッパー

海外人気

- ・ My Batis: SQLマッパー

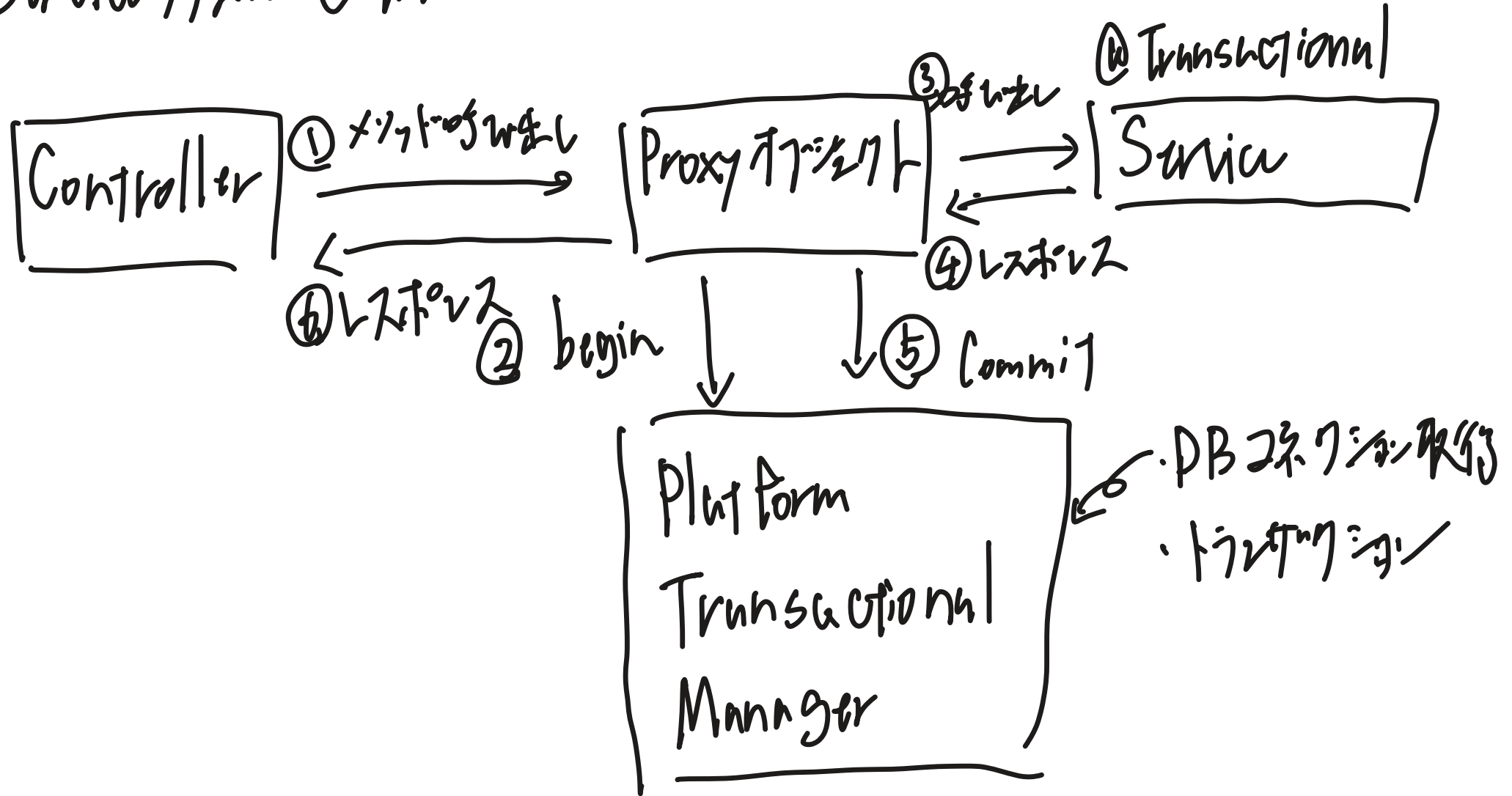
国内人気

- ・ JDBC Template: JDBCのマッパー

業務では使われる

• @Transactional の仕組み

Service 側で @Transactional を付けている場合



@Transactional 利用方法

- ・ Auto Configuration を利用 ⇒ 追加対応不要

- ・ Auto Configuration を利用しない

⇒ Configuration File (Java Config)

@EnableTransactionalManager の付加が必要

• Proxy パターン

- @Transactional or 何かエントリメソッドを継承したメソッド

||

付加されたメソッドと同じメソッドを持つ

- Spring は Proxy パターンを Bean として扱う

||

Service メソッドは Bean として扱われる

- Controller から Service メソッドを呼び出した際.

実際に呼び出されるのは Proxy パターン

- PlatformTransactionManager を呼び出して
トランザクションの取得とトランザクションを開始する

・ Proxy オブジェクトの役割

⇒ Service 712 処理する前に

Platform Transactional Manager を経由させる

⇒ Service 712 処理時に

トランザクション取得, トランザクション開始時の状態に遷移

Platform Transactional Manager の役割

① コネクションの作成

作成したコネクションは Thread Local に保存

② トランザクションの管理

Platform Transactional Manager 利用方法

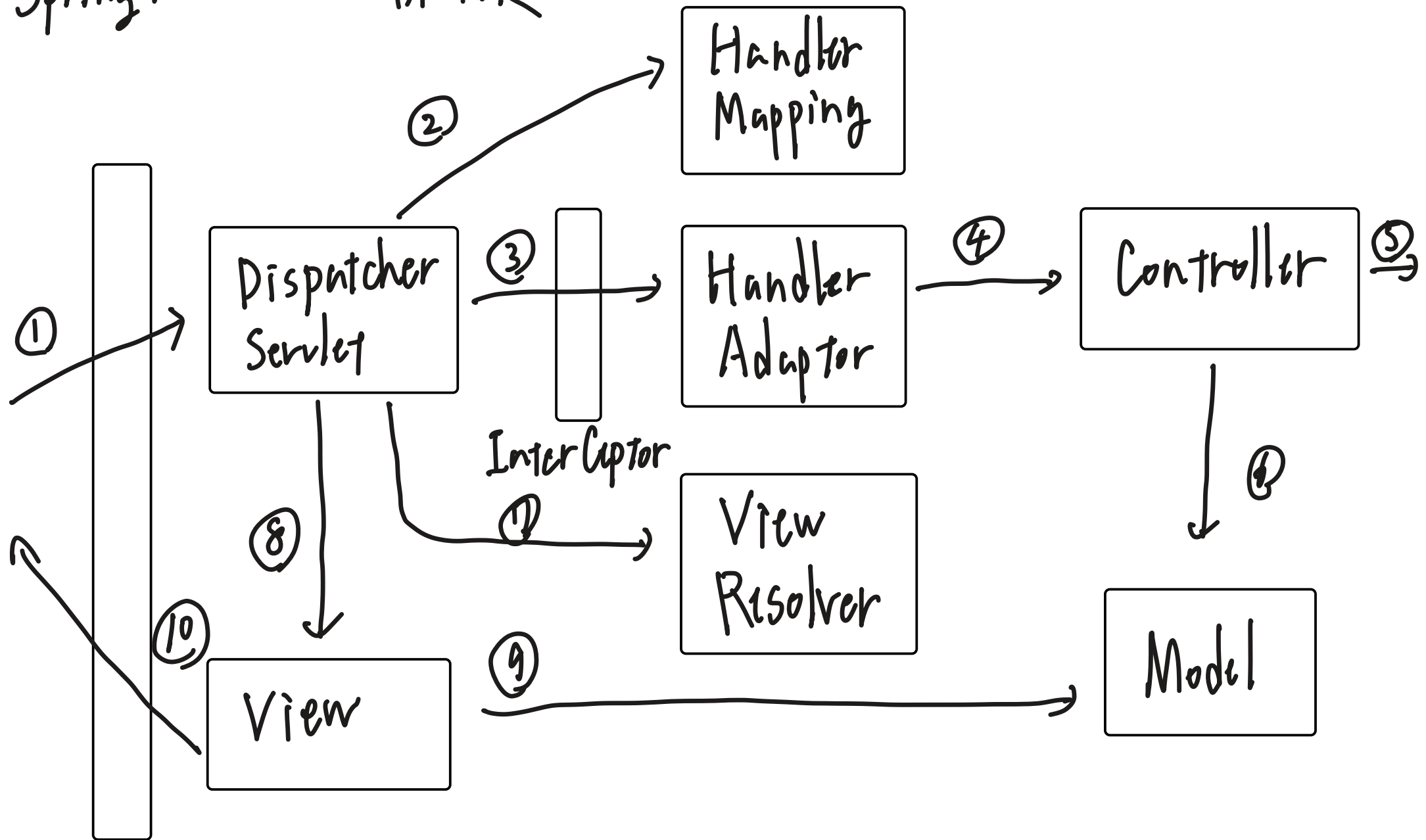
Bean 生成に

Auto Configuration を利用する $\Rightarrow$ Bean 定義不要

Auto Configuration を利用しない $\Rightarrow$ Bean 定義必要

$\hookrightarrow$ DataSource 必要

Spring MVCの全体像



Servlet Filter
(Security Filter Chainを含む)

- Dispatcher Servlet

- Spring Boot アプリケーションのフロントコントローラー
 - 全てのリクエストを一括受信し適切な部品への交通整理

@Controller を付与したクラスは Dispatcher Servlet 前
前処理をした後の処理を担当している

・Xリクエスト

- URLルーティングを行う
- リクエストの解析を行う

- Handler Mapping

リクエストされたURLからこのコントローラのメソッド(ハンドラー)を
処理するものを特定する

- ・ Handler Adapter : コントローラーの実行に引数の準備
 - ・ Handler Mapping で見つけたメソッドのハンドリングを呼び出す
 - ・ 呼び出す前に、ハンドリングの引数をリクエストデータの紐づけを行う
 - ・ @Validated がある場合、バリデーションを行う
 - ・ @RequestBody と @ResponseBody がある場合、
HttpMessageConverter (Jackson) からリクエスト・レスポンスを変換する

- ・ View Resolver : Viewを用意

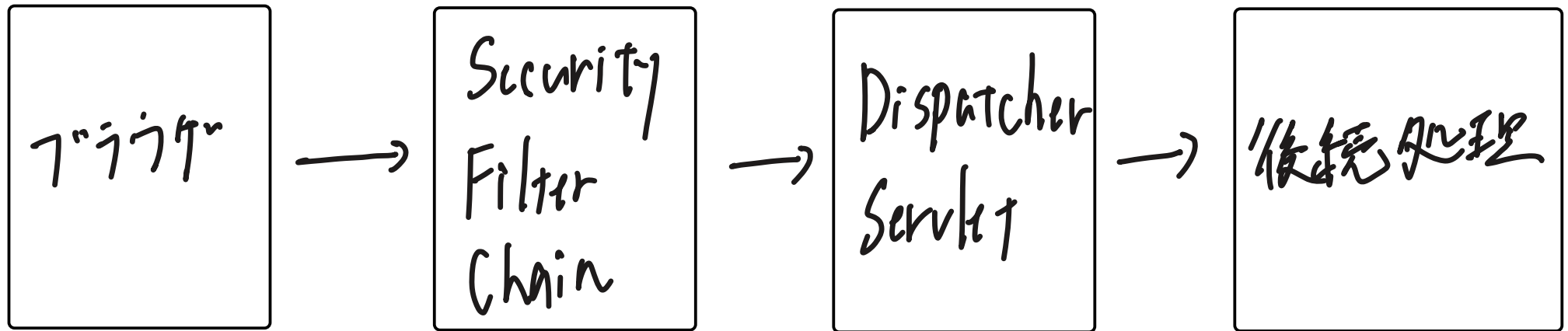
- ・ Viewを用意して Thymeleafなどのテンプレートエンジンへ
処理を引渡し

- Spring Security

代表的な認可

- ・ リクエストの認可 (アクセス可否)
- ・ メソッド認可 (呼び出し可否)
- ・ 画面表示認可 (一画面の中で表示し得る部分を制御)

Spring Security Filter Chain



↑ 認証Filter

・認可Filter

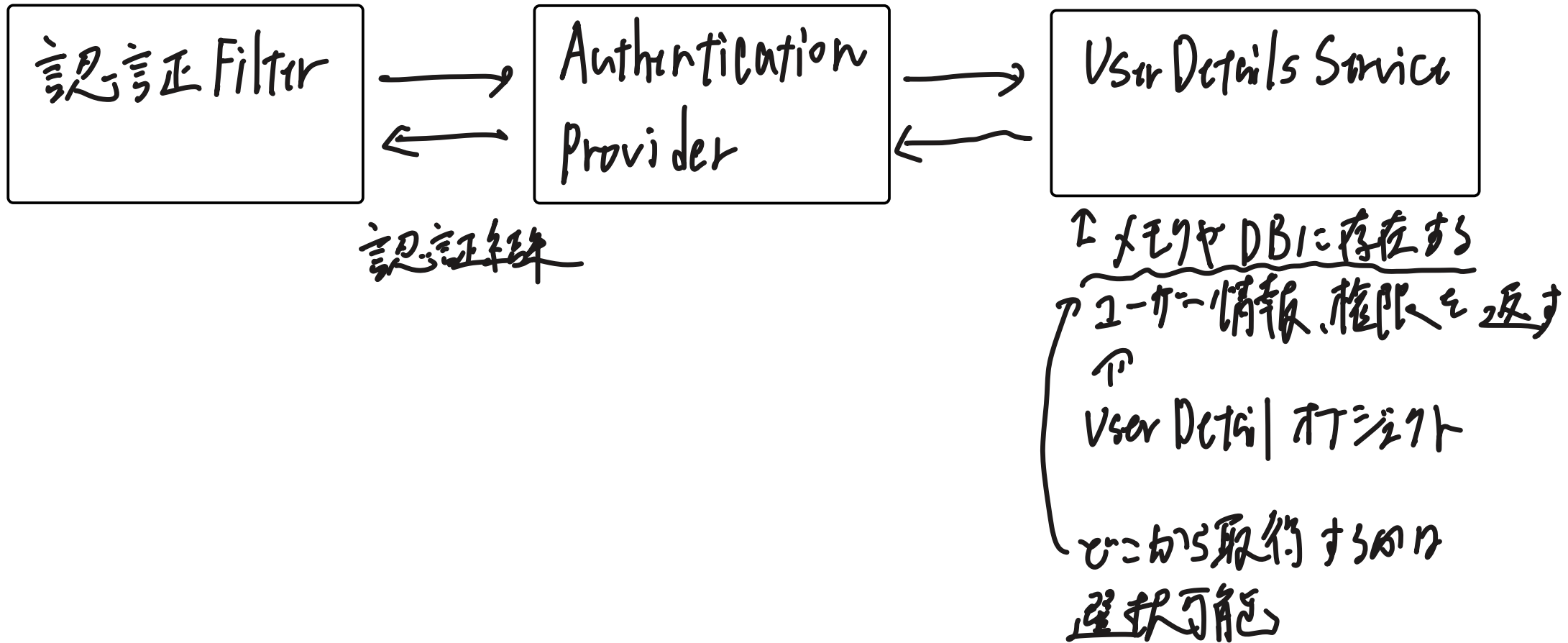
・例外ハンドリングFilter

※ Security Filter Chainの例外をハンドリングする

Security Filter Chain 利用方法

- ・ Java Config で Security Filter Chain の Bean 定義を追加し、
このような Filter を設定するのを定義する
- ・ Bean 定義方法: Java Config に `@EnableWebSecurity` を付ける

Spring Security 認証

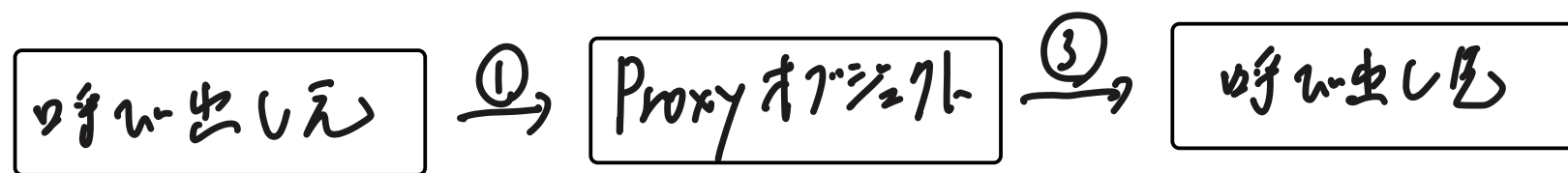


Spring Security 画面表示の認可

Thymeleaf で判定する (Spring Security の SpEL を利用)

メソッドの認可

- ・ Java Config に `@EnableMethodSecurity` を付加
- ・ 認可を行うメソッドに `@PreAuthorize (条件)` を付加
→ 対象メソッドが呼び出される前の Proxy オブジェクトの判定

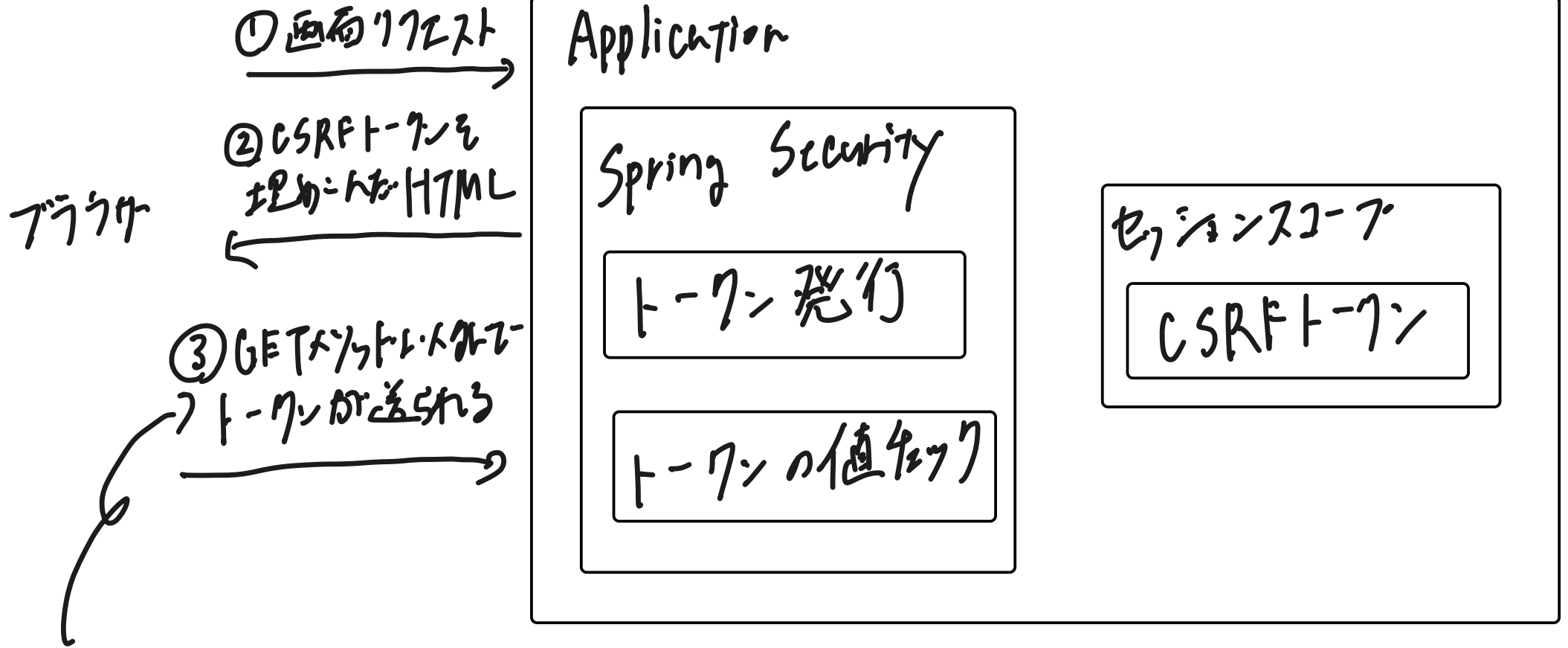


↓ ② 認可判定依頼

Authorization
Manager

CSRF対策

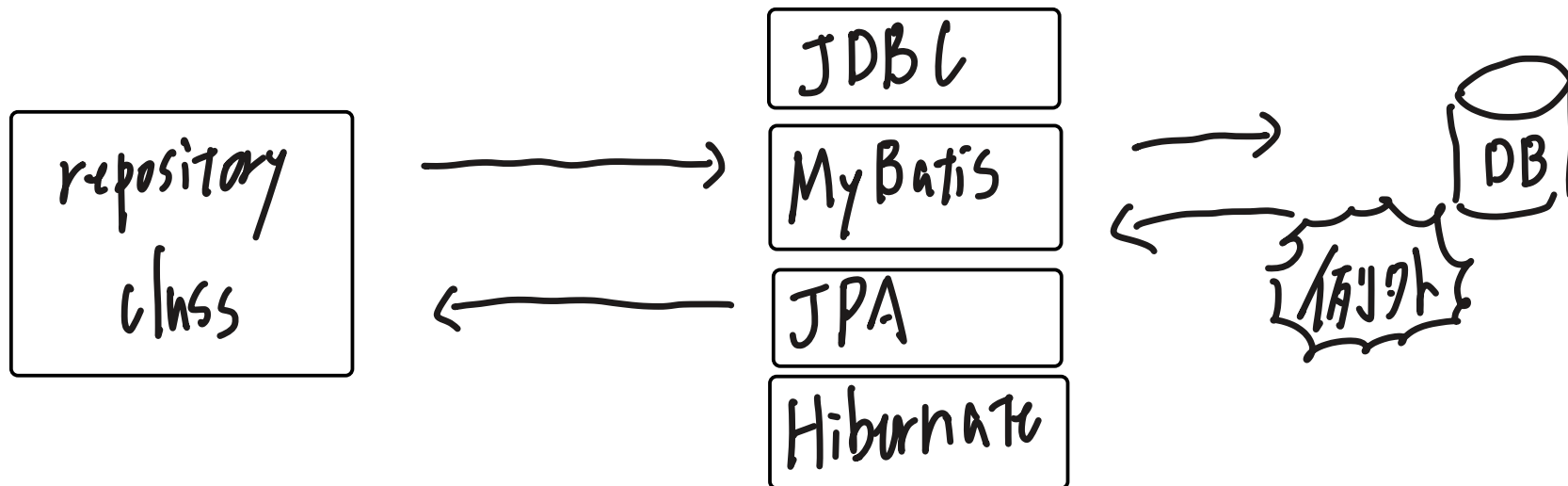
Spring SecurityのCSRFトークンの発行と値をGETする



リクエストするFormは

th: action属性を利用する必要がある

- ・ データベースアクセス時の例外
spring を利用しない場合



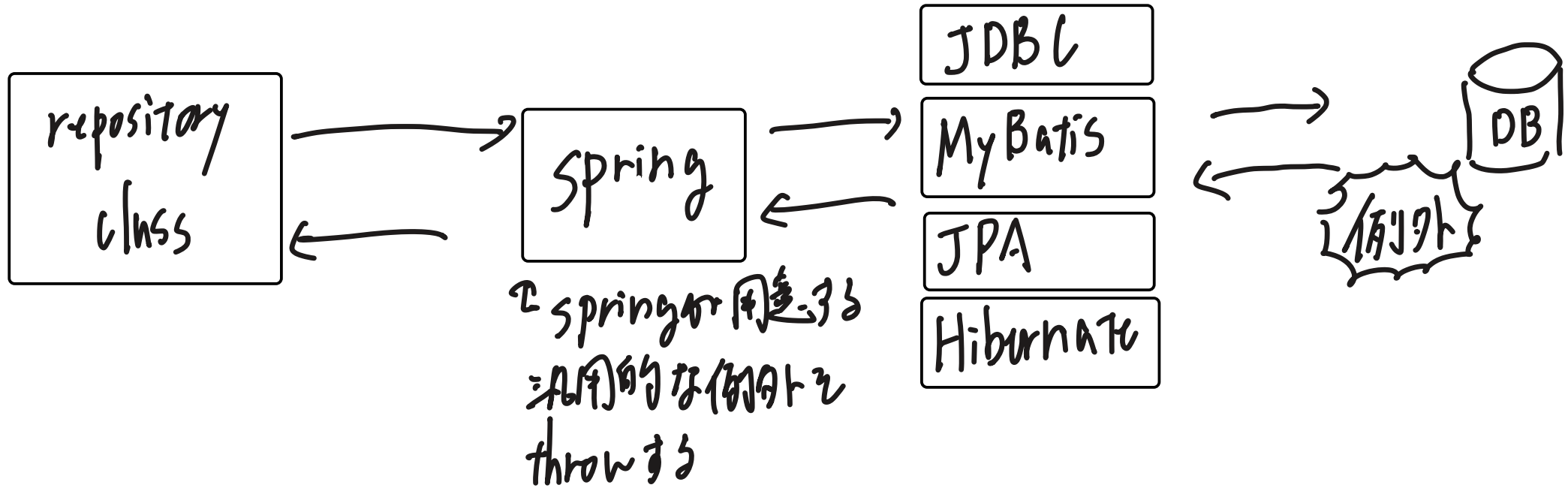
利用されるライブラリによって
throw される例外が変わる

=> catch する例外が変わる

=> ライブラリに依存したコードになる

データベースアクセス時の例外

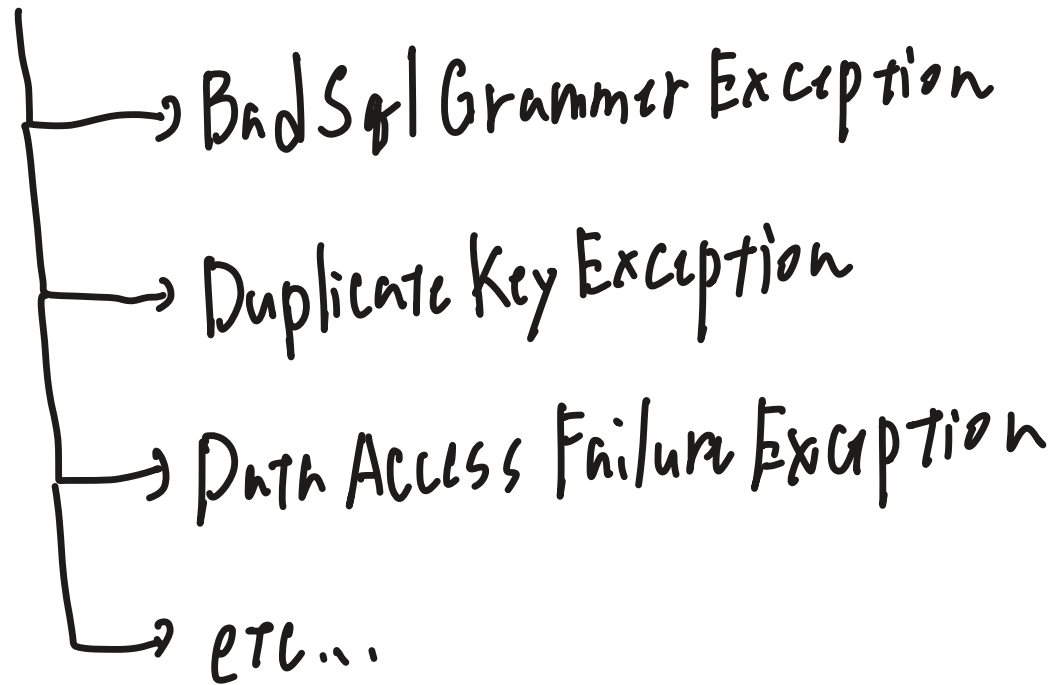
- Spring を利用する場合



⇒ ライブラリに依存しないコードになる

⇒ 利用するライブラリを変更するのが
容易になる

- SpringがthrowするDB系のException
 - Data Access Exception



※ 全2非空の例外 = 持つ位置を知らぬ

Transaction の伝播

Foo Service
@Transactional
foo Method()
barService.barMethod



Bar Service
@Transactional
bar Method

Transaction の伝播

Transaction

⦿
(上記処理の場合)

or

Transaction

+

Transaction

⦿ 各処理毎に
Transaction のコミットを行う
⇒ データ不整合が
起こる可能性がある

Transactionの伝搬の特徴

- ・ 全ての処理が終わったタイミングでコミットする
⇒ 一つでも処理が失敗すると全てロールバックする



Transactionの伝搬を同一な場合

- ・ 各処理でTransactionを取得し、
各処理が終わったタイミングでコミットする
⇒ 各処理毎にデータを登録出来る。
ただし、データ不整合になる可能性がある。

Transactionの伝播を利用するのびじり

@Transactionalのpropagation属性の値で切り替える

- ・ REQUIRED: 伝播する (デフォルト)
- ・ REQUIRED_NEW: 伝播しない

Transactionの伝播を利用しない時の実装の注意点

↳ 同一クラス内の別メソッドを呼び出す場合は、

propagation属性に REQUIRED-NEWを指定しても必ず伝播する

Foo Class

@Transactional

foo()

def();

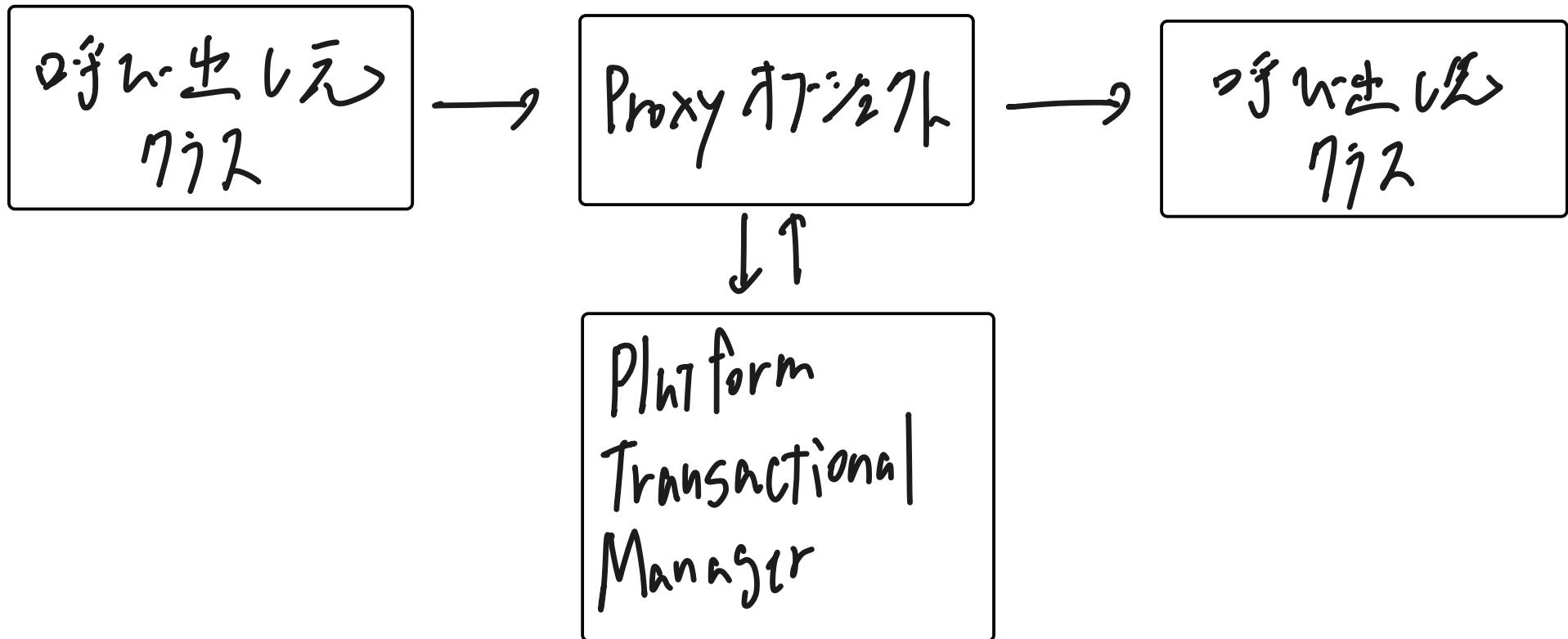
@Transactional(propagation=REQUIRED-NEW)
def()

Transactionの伝搬が必ず発生する理由

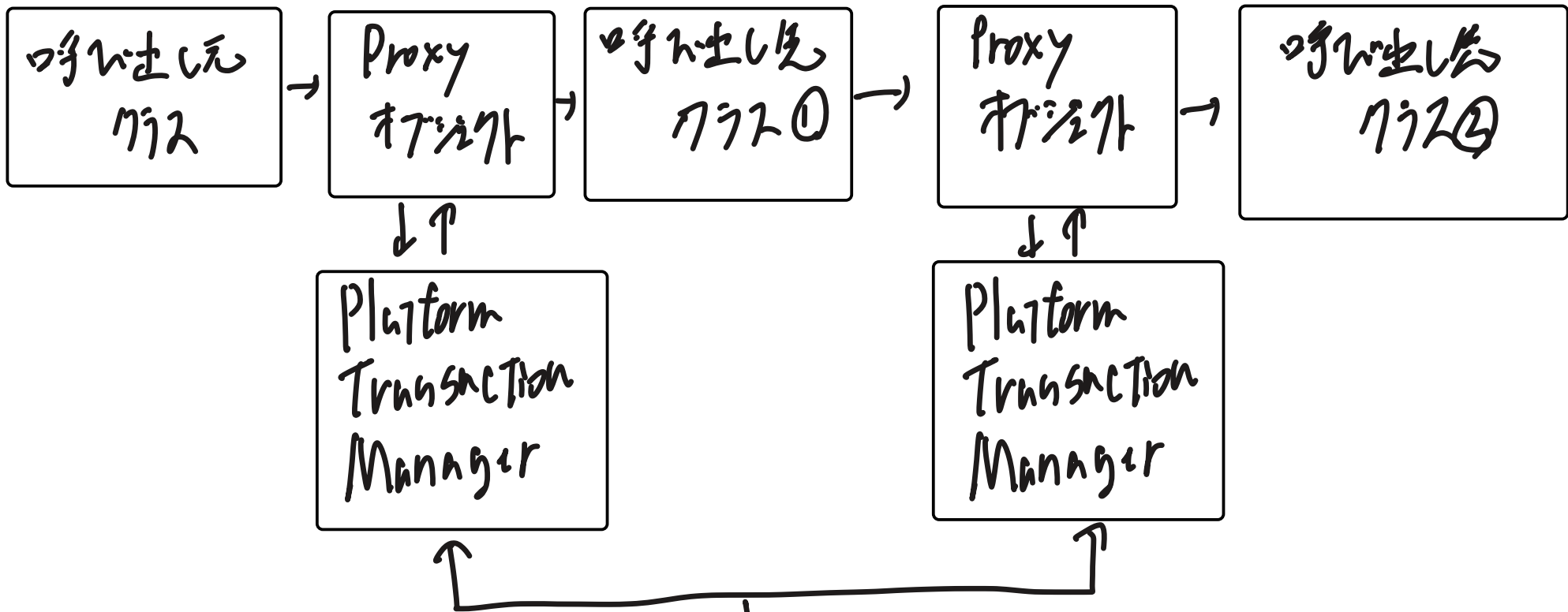
Transactionの管理は @Transactional が定義されているクラスの

Proxy オブジェクト (正しくは Platform Transactional Manager) が行う。という

⇒ 同一クラスの場合、1つの Proxy オブジェクトが Transaction の管理を行うため、必ず伝搬する



伝搬しない時



各クラスのTransactionを管理する
=>伝搬している分、新しいTransactionを作るの
は必要

・セッションスコープ

⇒ブラウザが毎に保存したデータを保存出来る領域を
(X)セッションカートなど

・セッションスコープのBean

・ @Session Scope を付加する

↳ 上記のステレオタイプアノテーションを付加すること

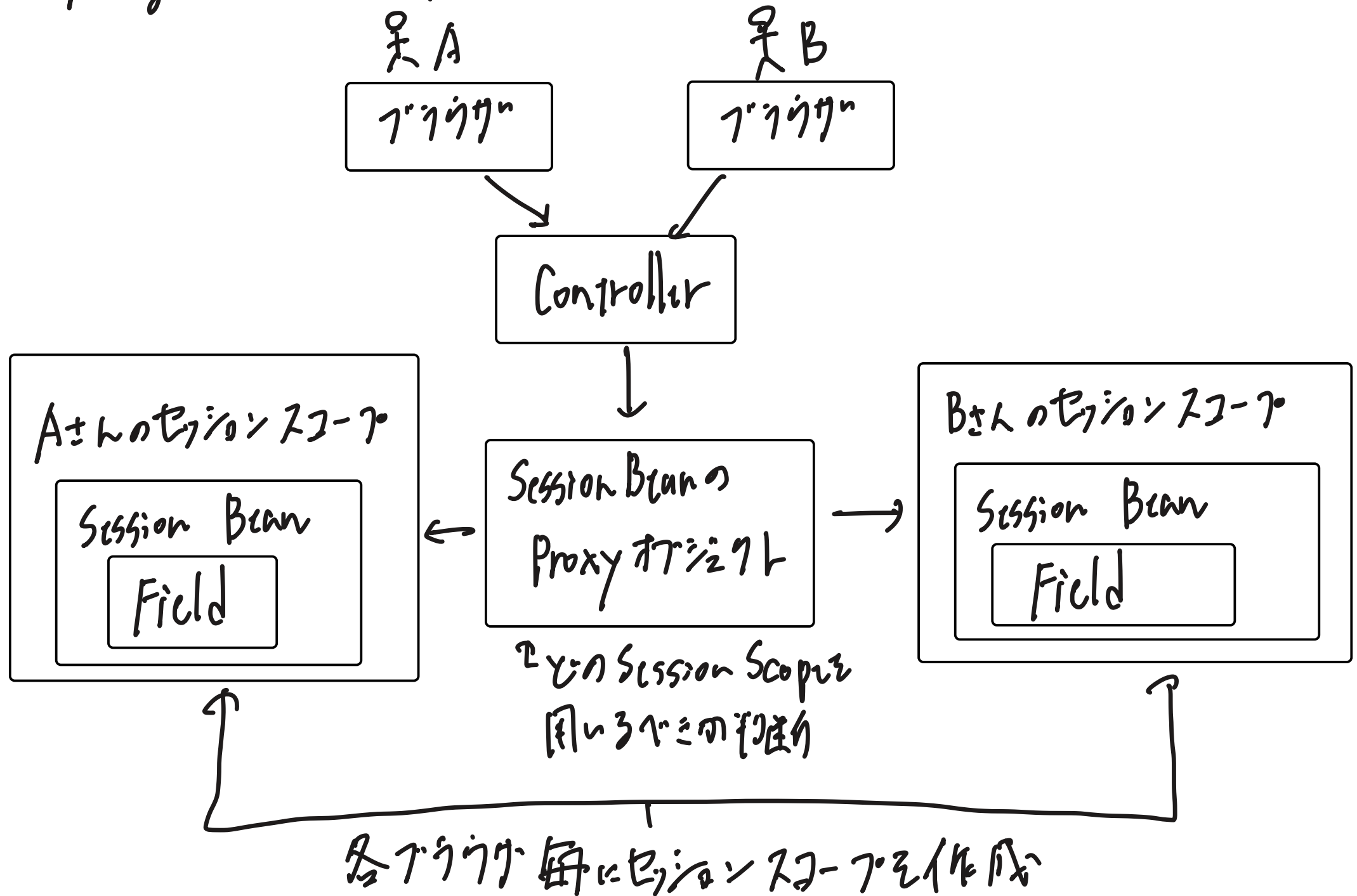
Bean対象であることを (Component Scanの対象であること) を明示する

・ implements Serializable をつける

※セッションスコープ: 保存するにはシリアライゼーションが必要

※ @SuppressWarnings("serial") をつける

Spring の Session Scope Bean

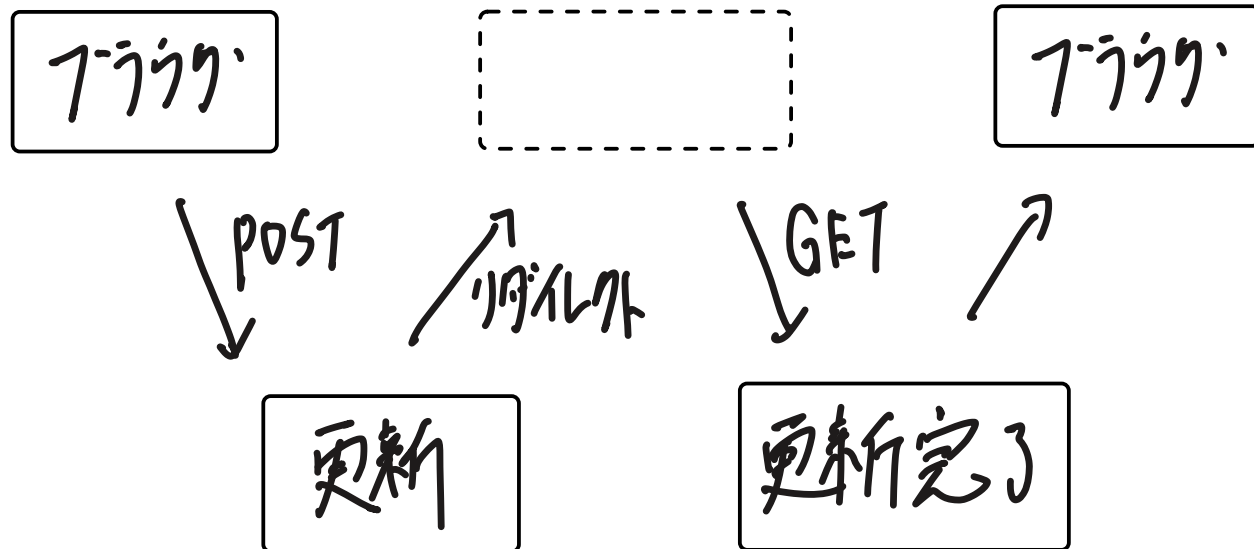


Flash Scope

・リダイレクト元からリダイレクト先にデータを運搬するための領域

・リダイレクトの利用シーン
更新処理の場合

PRGパターン



更新完了画面を

リポートしても更新完了画面が
再表示されるだけ。

更新処理は呼び出されず
データ整合性(多重リクエスト)が
発生しない

・リダイレクトの問題点

データ連携方法

- ・ Model: リクエストスコープのため、リダイレクト先へ連携出来ない

- ・ Session: リダイレクト先で利用可能

ただし、Sessionは明示的に破棄しないと

メモリに貯まり続けるため、適切なタイミングで

破棄が必要

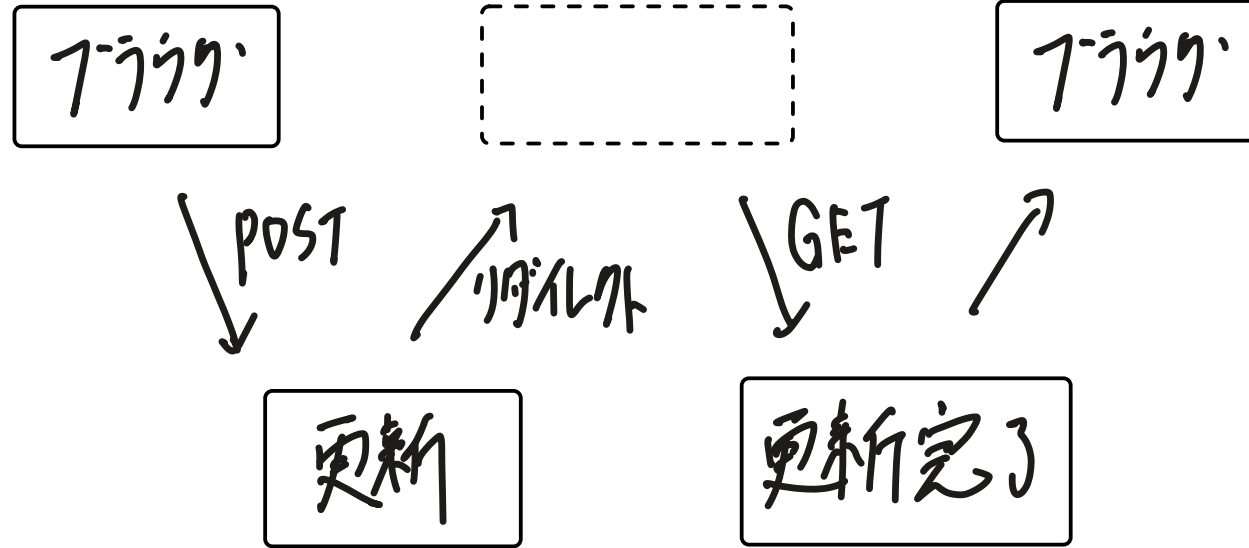
※リダイレクト先表示時に破棄するのは

破棄して良いタイミングから遠いので非推奨

★ フラッシュスコープ: リダイレクト先へ連携可能で

リダイレクト先表示時にデータを破棄が可能

🔗 スコア



リクエスト スコア

アップロード スコア

セッション スコア

Spring Security の認証・認可したユーザー情報

↳ Security Context に格納

- ・ 格納内タイミンク : Security Filter で認証が出来る時
- ・ 格納先 : セッションスコープ ... 認証済みのユーザを確認する場所
Thread Local ... 実行処理が必要時の取得場所

プロパティの定義場所

- ・ OS の環境変数
- ・ システムプロパティ (java コントロール)
- ・ プロパティファイル (.properties)

プロパティの読み込みと格納先

- ・ 環境変数
- ・ システムプロパティ
- ・ プロパティファイル

} DIコンテナ生成時
自動的に読みこまれる

} spring boot の
オートコンフィグレーション

格納先: Environment オブジェクト

- ・ 独自のプロパティファイル (application-プロパティファイル) を
記述する方法

- ・ Java Config に @PropertySource (プロパティファイル) を利用

- ・ Java Config の @Profile をつけることで切り替えも可能

- ・ Spring boot を利用する場合

起動するプロパティファイルのプロパティファイル + application.properties を記述

→ Spring 起動時の Java コードで指定

プロパティファイルを指定せずに起動した場合、プロパティ: default が適用される

プロパティファイルが複数ある場合はプロパティファイルの指定優先度

・プロパティの取得方法

@ Value: Environment オブジェクトから直接取得

@ Configuration Properties: prefix から同一のグループを

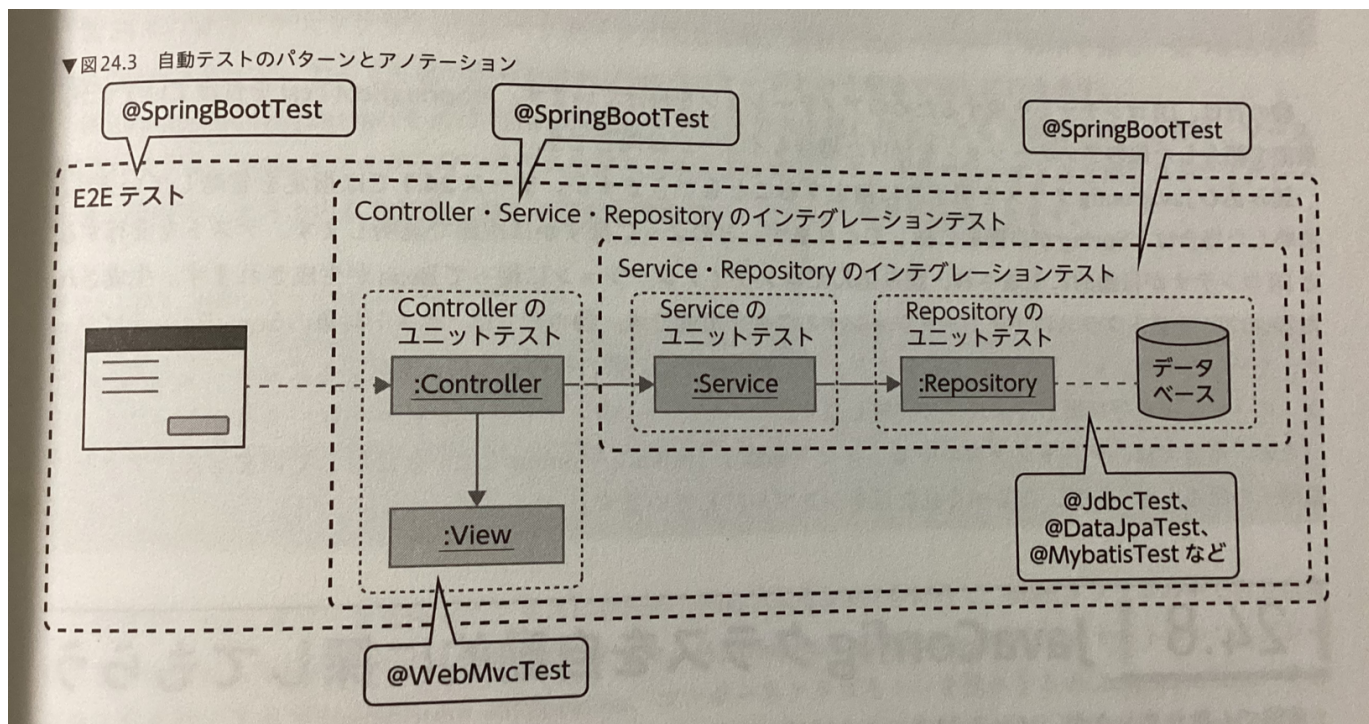
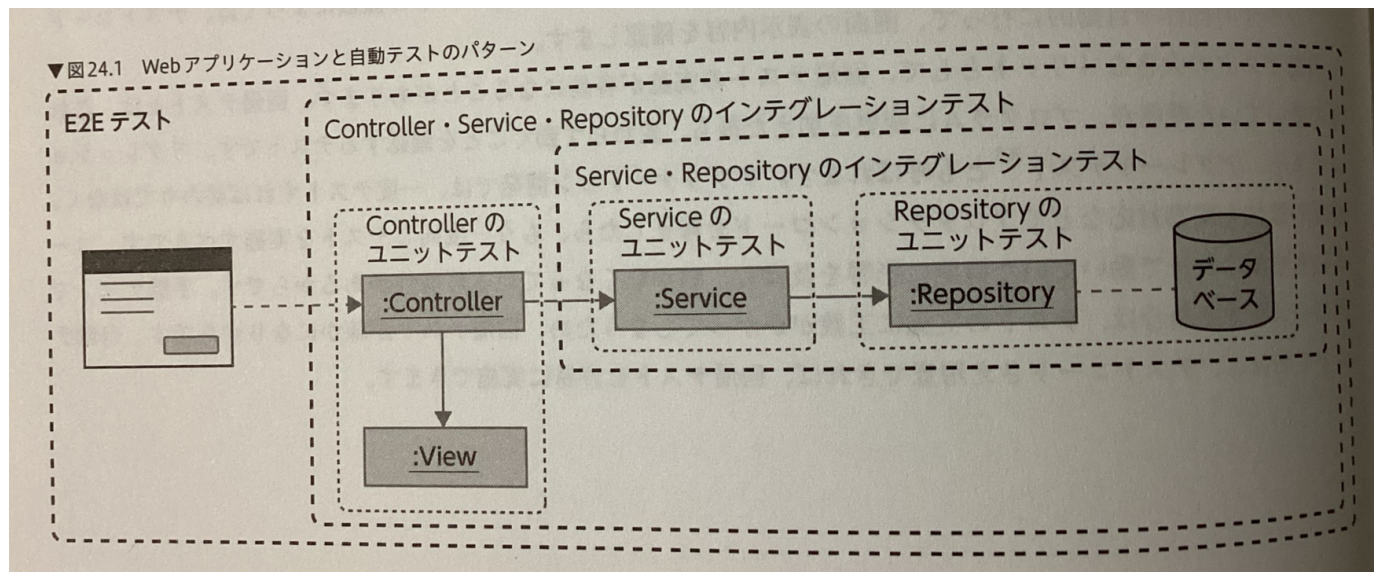
管理するリストを作成し、その中で取得する。

④ @Component などの Bean として扱う

⇒ DIコンテナ生成時に値が格納される。

インジェクションが可能

テストの考え方 (全体像)



テスト

- ・ Service 712 のテスト時は、DIコンテナを作らない ← Spring の機能を使わないため
(単体)
- ・ 上記以外の場合は、DIコンテナを作り、テストする
- ・ @WebMvcTest などを利用して、不要な Bean を作成しない
⇒ 時間を短縮

テスト時の DIコンテナ作成時の注意点

DIコンテナ作成は Java Config 元で Bean を作成することもある

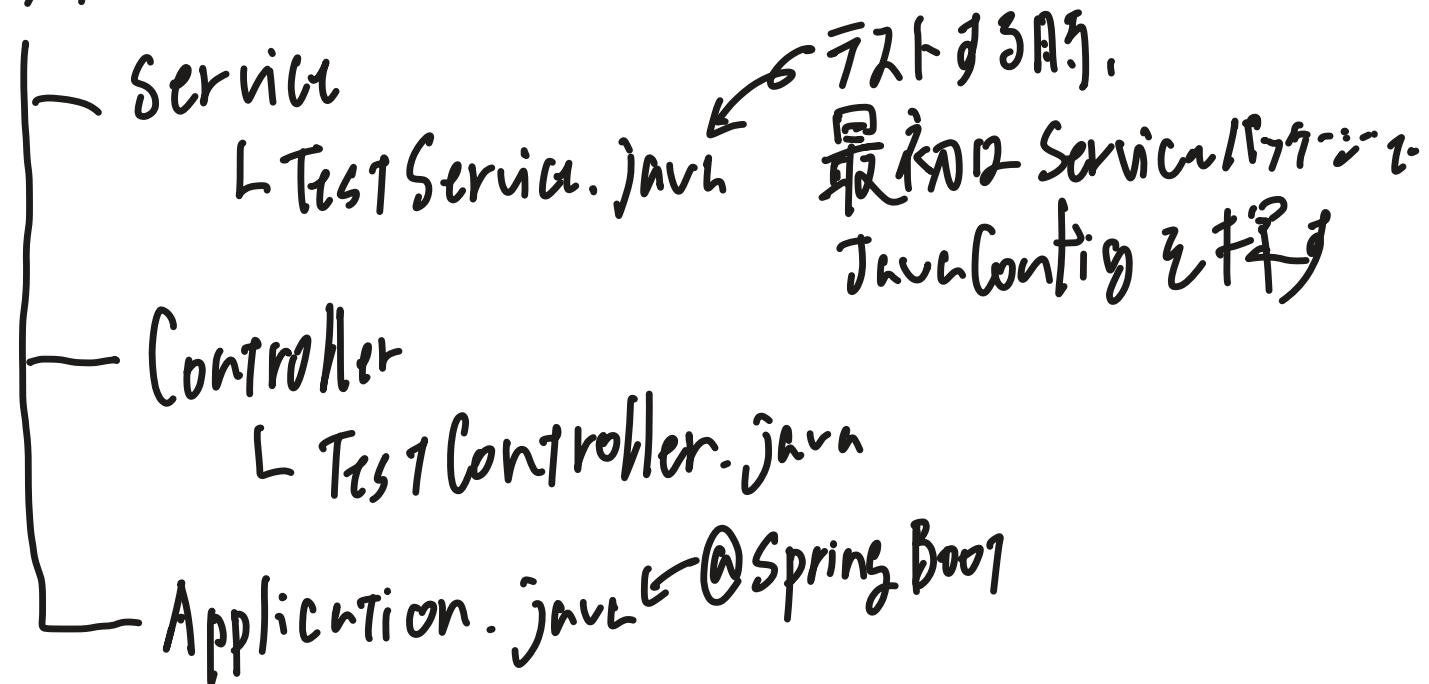
⇒ Java Config を採用する必要がある

※ 採り方

テスト対象のファイルのパッケージで Java Config を採用

無い場合は上の階層で採用、を繰り返す

例) src/main/java/パッケージ

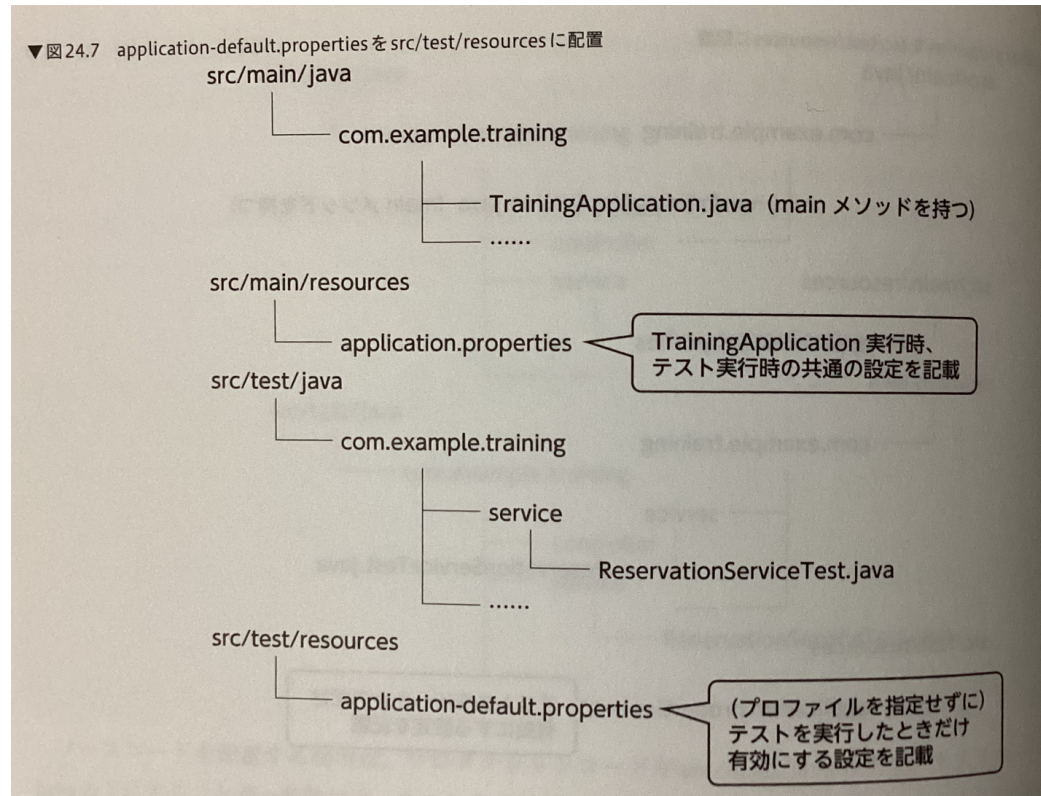


テスト実行時のみ利用するプロパティ

- src/test/resources
 - ↳ application-default.properties

- src/test/resources のプロパティファイルはテスト実行時しか利用されない

- src/main/resources のプロパティファイルも利用したい場合、
以下構成にする



Repository のユニットテスト

7/2: ① Jdbc Test. ② JPA Test など

③ SGL など (テストケースの投入)

7/2 はなく

7/2 に合わせて

① Jdbc Test の特徴

- コンポーネント スケインの範囲を制限する

⇒ ① Repository の Bean も作成しない

そのため、自身でテスト対象のインスタンスを作成する必要がある

※ オートコンフィグレーションは行わないため、DI コンテン Spring の提供する Bean は
型解決される

② Import (テスト対象のクラス) の Bean 生成は出ないが、テストに必要とする Bean は
理由) Spring のテストサポート機能は同一コンフィグレーションで実行される。

DI コンテンをテストするため、同一コンフィグレーションで処理時間短縮。

③ Import を利用する別のコンフィグレーションでテストを行う。テストは実行される

・ Serviceのユニットテスト

前提) Serviceは業務ロジックのため、Springの機能を利用しない想定

⇒ DIコンテナは生成しない (オートコンフィグレーションやBean生成しない)

Serviceを利用する機能 (Repositoryなど) はMockする

⇒ Mockitoを利用する

・ Mockitoの使い方

- ・ 7人: @ExtendWith (Mockito Extension.class) を付加

- ・ テスト対象フィールド: @InjectMocks

- ・ モック対象フィールド: @Mock

Mockは

- ・ 戻り値の設定

- ・ 呼び出し回数

- ・ 引数を知り 実行する

・ Service, Repository のインテグレーションテスト

Repository 71スも含むため, DIコンテナの生成 + オートコンストラクションを含むBean生成が必要

ただ, Spring MVC (web周り) のコンストラクションは不要

そのため, テスト71スに以下のように記述

@SpringBootTest (WebEnvironment = WebEnvironment.NONE)

@Sql

@Transactional

↑ @SpringBootTest に含まれないため, 追加した方が良さ

※ @JdbcTest ならば @Transactional を含む

・コントローラーのユニットテスト

主目的: 指定したリクエストで対象ハンドラーメソッドが呼び出されていることの
確認

- ・バリデーションチェック

- ・レスポンスのHTMLに期待値が含まれているか (一部チェック)

※ Service や Repository はモックする

テスト実施

リクエスト: @WebMvcTest ← @Service なり Bean 生成しない

ファイル: MockMvc ← 擬似的にリクエストを送信, レスポンスのアクションが可能

モック: @MockBean: モックするリクエストを Bean として管理する (@Mock の違い)
裏では Mockito を利用しているため, モックの利用方法は @Mock と
同じ

Dispatcher Servletの主要コンポーネント

- Multipart Resolver: リクエストのボディのマルチパートの判定
- Locale Resolver: ブラウザや cookie の言語を認識
- Theme Resolver
- Handler Mapping: URL からコントローラを判定
- Handler Adaptor: Handler メソッドへ渡す引数の宣言とコントローラ実行
- Handler Exception Resolver: コントローラからの例外を適切なエラーレスポンスへ変換
- RequestToViewNameResolver: View が指定されていない場合、リクエストから判定
- View Resolver
- Flash Map Manager

Spring Boot 起動の裏側

・コマンド: `spring-boot:run`

↳ ① Maven プラグインの起動とゴール解決 (Mavenの世界)

② プロジェクトのビルドとリソースの構築 (依存関係・コンパイル)

③ 分離されたリソースの生成 (Spring Boot特有のメカニズム)

④ Spring IOCコンテナの起動と組み込みサーバーの立ち上げ
(Springの世界)

Mavenプロジェクトの起動とゴール解決

- ・ Mavenはプロジェクトのpom.xmlを読み込む

↳ 「spring-boot-maven-plugin」のrunゴールを見つけ実行。

通常のMavenビルドはソースコードをコンパイルしてJarやWarという

アーティファクトをターゲットディレクトリに出力して終わるが、

spring-boot:runは出力するだけでなく、

そのまゝJVMプロセスとして起動するというゴールも存在する

※ runゴールではJAR・WARファイルは出力しない(スキップ)
コンパイルしてワイルドファイルは出力する

mvn spring-boot:run の実行

⇒ Maven は プラグイン に指定されたゴールをポイントで実行する

※ Mavenビルド全体が最初から最後まで実行されている

pom.xml |

dependencies: 外部ライブラリ

build: ビルドプロセス (コンパイル, テスト, パッケージ) や使用するプラグイン, 出力先を定義

↳ plugins: 何から何まで出力する方法を定義している (ゴール)

Mavenのビルドについて

・ 最終目的: ソースコードを求むた形式で出力や起動する (JARファイル出力やアプリ起動など)

・ 手段: ① Mavenで定義されているビルドプロセスを組み合わせて自作する
※ 基本自りに任せる

② plugin の ゴール を用いる

↳ ビルドプロセスをまとめたもの

ゴールを呼ぶたびに定義されているプロセスを実行させる

mvn spring-boot : run
↑ プラグインを指定 ↑ プラグインのゴールを指定

build
└ plugins
 └ plugin
 └ spring-boot-maven-plugin
 └ goals
 └ run ———— <ビルドゴール>
 └ repackaging
 ① コンパイルする
 ② 実行可能ファイルに生成する
 ③ Tomcat に組み込む

Tomcatの起動

→ JVMのプロセスで起動する

※ spring-boot: runモードの場合、

コンパイルしたクラスファイルを用いて、Tomcatは起動し、

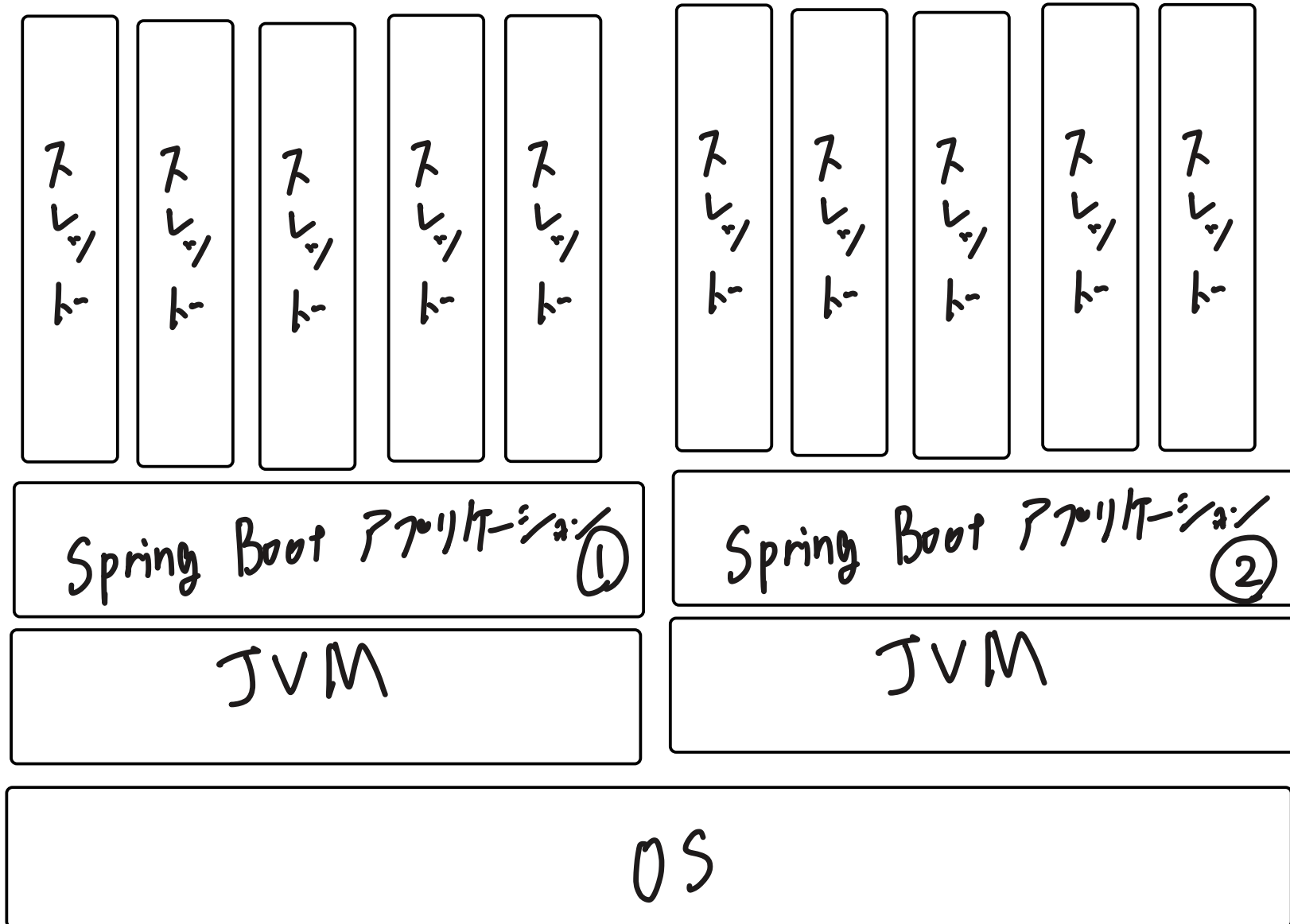
起動する際、JVMプロセス上で起動する

→ Spring Bootアプリ (= JVMプロセス) の内部のライブラリ・スロットで起動している

JVM

- ・ 1つのJVM = OSの1つのプロセス
 - ⇒ 1つのJVMで複数のJVMプロセスは起動出来ない
- ・ 1つのJVM毎にメモリは割り当てられる
- ・ 1つのJVMは 1つの Docker コンテナの様なイメージとして提供される

・ Spring Boot 起動時のイメージ



依存関係の解決とクラスパスの構築

コードを実行するため、自分たちが書いたクラスだけでなく、

Spring 本体や Hibernate, Tomcat などの外部ライブラリ (JAR) を

JVM に渡す (クラスパスを通す) 必要がある

↳ pom.xml に定義されている依存関係をマニフェストから取得する

spring-boot:run は, src/main/java か src/main/resources から

コンパイルした直後の状態 (target/classes) のまま, 外部 JAR と

組み合わせて起動出来るように準備する

実際の発動では mvn spring-boot:runを実行すると

①依存関係の解決

②ビルド フォウゲインのゴールの実行

という順番で処理される

依存関係の解決

mvn spring-boot:runを実行するとまず依存関係の解決が行われる

pom.xml

dependencyで定義している外部ライブラリのJARファイルを

Mavenのローカルリポジトリやインターネット(Maven Central)から取得する

※ 外部ライブラリが必要とするJARファイルも取得する(等々)

⇒ 依存関係が解決されると、外部ライブラリのJARファイルが

ローカルリポジトリに全て存在するようになる

※ 関連するJARファイルも含めて

クラスパスの構築

→ ファイルやクラスを探索に行くための検索ルートのリスト

(ディレクトリや JAR ファイルのパス)

依存関係の解決が終わっている = クラスパスの構築に必要な情報は揃っている

⇒ Maven はクラスパスの構築を行い、JVM にクラスパスを渡す

※ ファイルやクラスを探索するのは JVM で、JVM はリストの順番대로探索していく。

専用クラスローダー (Launched URL Class Loader) の生成

- ・ Spring は、Java 標準のクラスローダーではなく Spring boot が独自に拡張したクラスローダーを JVM 上で生成する

↳ 標準のクラスローダーでは JAR を直接読みこく

- ・ 通常の Flat JAR (すべての依存ライブラリを 1 つの JAR にまとめる)
- ・ JAR ファイルの中にある別の JAR ファイル (NESTED JAR)

Spring Boot 独自のクラスローダー

- ・ クラスパス内の JAR ファイルをディレクトリを再帰的にスキャンし標準ではロード不可能な構造の依存関係をメモリ上にロード出来る

クラスローダー

処理が済んだ後、必要な Java クラスオブジェクトを JVM のメモリ上に場合、
コンパイルされた class ファイル (バイトコード) を JVM のメモリに読み込んで
Java クラスオブジェクトに変換する

実際に行っていること

- ① ロード : 上記のメモリに読み込むための作業
- ② リンク : メモリに読み込んだクラス同士の間参照先が合っているか
※読み出した後に X/7トは存在するの、型が合っているか否か
- ③ 初期化 : 読み出し可能な状態にする
※この時、インスタンス化はしていない。クラスオブジェクトの内部
Static の値の設定が必要

クラスローダーの階層構造

1. ボートストラップ・クラスローダー

↳ JVMの標準ライブラリをロードする

2. アプリケーション・クラスローダー

↳ 標準拡張ライブラリをロードする

3. アプリケーション・クラスローダー

↳ target/classes と Mavenで取得した外部ライブラリのJARファイルをロードする

4. LaunchedURLClassLoader (Spring特製のカスタムクラスローダー)

↳ Springのバッチ処理構造や組込みWebサーバーを呼び出す仕組みに

対応するために存在する。

ネストされたJARファイルをロードする

Launched URLClassLoader

・ Spring Bootは「Fat JAR (またはExecutable JAR)」という特殊なパッケージ形式

・ 課題: JARファイルの中に別のJARファイルが入っている
(ネストされている状態)

⇓

通常のアプリケーション・ワウスケ-ターでは読み取れないので
Launched URLClassLoaderで読み取った。

※ アプリケーション・ワウスケ-ターは

target/classesに独立したJARファイルは読み取る。

ネストされたJARファイルは読み取れない

Spring IOCコンテナの起動と組み込みサーバーの立ち上げ

- ・ @SpringBootApplication のついたクラスの main メソッドからリフレクショを経由して起動される

1. Spring Application の初期化

Spring Application.run() が実行され、環境変数やプロパティの読み込み

2. IOCコンテナの構築 (Application Context)

・ @Component Scan が実行され、アプリケーションから読み込まれる Bean 定義の読み込み、DI のグラフをメモリ上に構築

3. Auto Configuration の実行

Spring Boot が必要な Bean を自動構築する

(クラスパスに存在する内容で生成)

4. 組み込みサーバーコンテナの起動

Tomcat などの組み込みサーバーが指定されたポートで起動する

コンソールに Started Application in X.XX second が表示される

④ Spring Boot Application のついでにリフレクション経由で呼び出す

- ・呼び出し元: spring-boot-maven-plugin の runゴール

- ・処理: ビルド後に、ビルド成果物をスキャンし、mainメソッドを持つクラスを探し、
リフレクションを用いて mainメソッドを実行する
(Javaの仕組み)

mainメソッドが実行された後の処理

① IOCコンテナ(DIコンテナ)を構築する

② インスタレーションファイルを読み込む

※ @Configurationがついているファイルを読み込む

@SpringBootApplicationは@Configurationを含む

※ インスタレーションファイルに@Beanメソッドがある。インスタンスを生成する

③ コンポーネントスキャンをする

※ @ComponentScanがついているパッケージの下の
コンポーネントスキャンの範囲になる

※ @SpringBootApplicationは@ComponentScanを含む

④ @SpringBootApplicationの@EnableAutoConfigurationを読み込む
オートコンフィグレーションが実行され、IOCコンテナに格納される

オートインストールの裏側

@Conditional On Class を使用

↳ 条件が揃った場合は自分で設定を有効にする

例: Tomcat の 自動起動

条件: JVM が読み込めるクラスパスに以下が存在する場合,

Tomcat を立ち上げる

- ・ org.apache.catalina.startup.Tomcat
- ・ jakarta.servlet.Servlet

■ 実行の流れ

- ① mavenがpom.xmlのdependenciesをもとに依存関係を解決 + コンパイル
- ② mavenはクラスパスを構築し、JVMへクラスパスを渡す
- ③ spring-boot-maven-plugin がリフレクションを用いてmainメソッドを実行
- ④ IoCコンテナが構築される
- ⑤ コンフィグレーションファイルを宣言する
- ⑥ コンポーネントスキャンが実行される
- ⑦ オートコンフィグレーションが実行される
 - ⑦-1 クラスパスから条件に合うクラスをいれるのステップする
 - ⑦-2 該当クラスをクラスローダーがJVMメモリにロードする
 - ⑦-3 ロードしたクラスをBeanとして生成し、IoCコンテナへ格納
 - (Tomcatの場合) ⑦-4 IoCコンテナに格納されたBeanのSTARTメソッドを実行してTomcatの起動

・ Spring Boot Dispatcher Servlet の起動タイミング

タイミング: オートコンフィグレーション実行時

↳ Spring MVC の仕組みを自動で整えるワザがある場合、

既に存在するワザは Bean に登録される

↳ Dispatcher Servlet が含まれる

⇒ その後、Tomcat が起動し、Tomcat にて

Dispatcher Servlet をサーブレットコンテナ (Tomcat) の メイン処理に設定する

※ Dispatcher Servlet 以外にも

Spring MVC の主要コンポーネント (HandlerAdapter, HandlerMapping など) は

オートコンフィグレーション時に Bean 登録されている

Spring Boot アプリケーションにおける Tomcat の役割

メイン: Java のプログラム (サーバレット) を動作させるサーバレットコンテナとしての機能

具体的な処理

1. HTTP リクエストの受信

2. サーバレットへのパス渡し

※ Spring Boot の場合, Dispatcher Servlet

3. HTTP レスポンスの返却

※ 基本的に 1 つ JVM では 1 つの Tomcat を起動させる

(やろうと思えば 1 つの JVM に複数の Tomcat を起動させることも可能)

1 つの JVM に 2 つ以上のアプリが停止するのはダメじゃない)

Tomcat

起動: 起動した環境の特定のポートへの通信を待ち受ける(リスン),
通信が来れば Tomcat が受けとって処理する

主要パーツ

- ① Coyote: 通信の受けと通訳
↳ 生データの TCP/IP や HTTP パケットを受けとって
Tomcat 内部で扱いやすい Java オブジェクトに変換する
- ② Catalina: サーブレットコンテナを実装に重なるメインエンジン
↳ Coyote から渡されたリクエストを
「このアプリのどのサーブレットに渡せば良いか」をルーティングして
ライフサイクルを管理する
- ③ Jasper: Jsp 用のテンプレートエンジン
※ Jsp は基本的に使われないため省略

Tomcat Catalinaのライフサイクル管理

- ・ サーブレットインスタンスのライフサイクル(起動・停止)を管理する

Spring Bootの場合)

サーブレットインスタンスはDispatcher Servletのみ

⇒ Dispatcher Servletのライフサイクルはアプリケーションの起動・終了と同じになる

純粋なサーブレットアプリの場合)

サーブレットインスタンスは複数になる

⇒ サーブレットインスタンスはリクエストが届いた際に起動(ポート)される

一定時間未使用や、プログラムで不用と判断したインスタンスは終了させる

Tomcatの処理過程

1. リクエストを受信するレリクエスト毎にスロットを作成する (スロットプール作成)
⇒ 並行処理になる

2. 割り当てられたスロット上で Coyoteが起動し.

ネットワークの生データを, Javaオブジェクト (HttpServletRequest等) に変換する

3, Catalinaがリクエストをルーティングし. Dispatcher Servletへオブジェクトを渡す
※ 割り当てられたスロット上で動作する

4. Dispatcher Servletが Spring 側の処理を行う

※ レスポンスは上記の逆順の流れることになる

SpringとTomcatを組み合わせたメリット

- ・Tomcatはリクエスト毎にスレッドを生成する

⇒ 並行処理が可能

- ・SpringはDTOはv:人などのインスタンスはBeanに登録する

⇒ 処理毎にインスタンス生成せず1つのインスタンスを使い回せる

⇒ 複数スレッドが基本的に1つのインスタンスを使い回すため

メモリの逼迫の防止や、インスタンス生成コストの削減が可能になる



IoC / シングルトン・スコープの強み!!

JVM

Tomcat

Coyote

Catalina

↓
Dispatcher Servlet

自作772

ブラウザ



